《数据结构》第四次实验报告

雷天乐 2025110528 25 数据科学 1 班

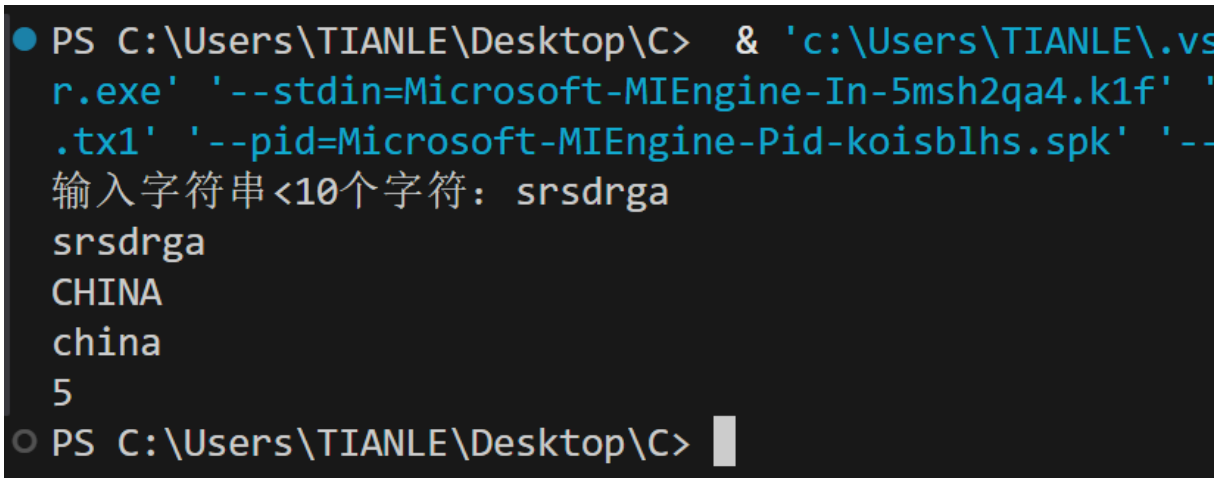
1 实验 1

实验任务： C 语言定义字符串数组
实验时间： 10 mins
知识点：本实验练习了 C 语言中字符数组与动态内存的使用。char a[10] 和 char b[]=”CHINA” 是编译器分配的字符数组，scanf(”%s”) 可从键盘读入字符串；malloc 函数可在运行时动态申请堆内存，配合 strcpy 可将字符串拷贝到该内存中，最后通过 printf(”%s”) 输出、strlen 求字符串长度。

1.1 源代码

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(){
    char a[10];
    printf("输入字符串<10个字符: ");
    scanf("%s",a);
    char b[]="CHINA";
    char *xx = NULL;
    xx = (char *)malloc(10);
// scanf("%s", xx);
// xx="china";
    strcpy(xx, "china");
    printf("%s\n", a);
    printf("%s\n", b);
    printf("%s\n", xx);
    printf("%d\n", strlen(xx));
    return 0;
}
```

1.2 输出结果



1.3 实验总结

通过本次实验，我掌握了 C 语言中字符数组的声明与初始化方法，学会了用 scanf 读入字符串、用 strcpy 拷贝字符串。同时理解了 malloc 动态内存分配的基本用法，能结合 printf 和 strlen 完成字符串的输出与长度计算，加深了对指针和内存管理的认识。

2 实验 2

实验任务： 定长顺序存储表示和串的联接操作
实验时间： 1 hours
知识点： 定长顺序串采用数组进行存储，SString[0] 用于保存字符串长度，其余单元依次存放字符。Concat 用于连接两个字符串，当总长度超过最大长度时会进行截断，并通过返回值判断是否发生截断。

2.1 源代码

```
#include <stdio.h>
#define MAXSTRLEN 255  // 用户预定义最大串长
typedef unsigned char SString[MAXSTRLEN + 1];
// 0号单元存放串的长度
#define TRUE 1
#define FALSE 0

typedef int Status;
Status Concat(SString T, SString S1, SString S2){
    int i;
    Status uncut;
    if (S1[0] + S2[0] <= MAXSTRLEN){
        // 未截断
        for (i = 1; i <= S1[0]; i++)
            T[i] = S1[i];
        for (i = 1; i <= S2[0]; i++)
            T[i + S1[0]] = S2[i];
        T[0] = S1[0] + S2[0];
        uncut = TRUE;
    }
    else if (S1[0] < MAXSTRLEN){
        // 截断
        for (i = 1; i <= S1[0]; i++)
            T[i] = S1[i];
        for (i = S1[0] + 1; i <= MAXSTRLEN; i++)
            T[i] = S2[i - S1[0]];
        T[0] = MAXSTRLEN;
        uncut = FALSE;
    }
    else{
```

```
        // 截断 (仅取 S1)
        for (i = 0; i <= MAXSTRLEN; i++)
            T[i] = S1[i];
        uncut = FALSE;
    }
    return uncut;
}


int main(){
    SString S1, S2, T;
    int i;

    printf("请输入第一个字符串: ");
    scanf("%255s", &S1[1]);

    printf("请输入第二个字符串: ");
    scanf("%255s", &S2[1]);

    /* 计算字符串长度 */
    i = 1;
    while (S1[i] != '\0')
        i++;
    S1[0] = i - 1;

    i = 1;
    while (S2[i] != '\0')
        i++;
    S2[0] = i - 1;

    if (Concat(T, S1, S2))
        printf("未截断\n");
    else
        printf("发生截断\n");

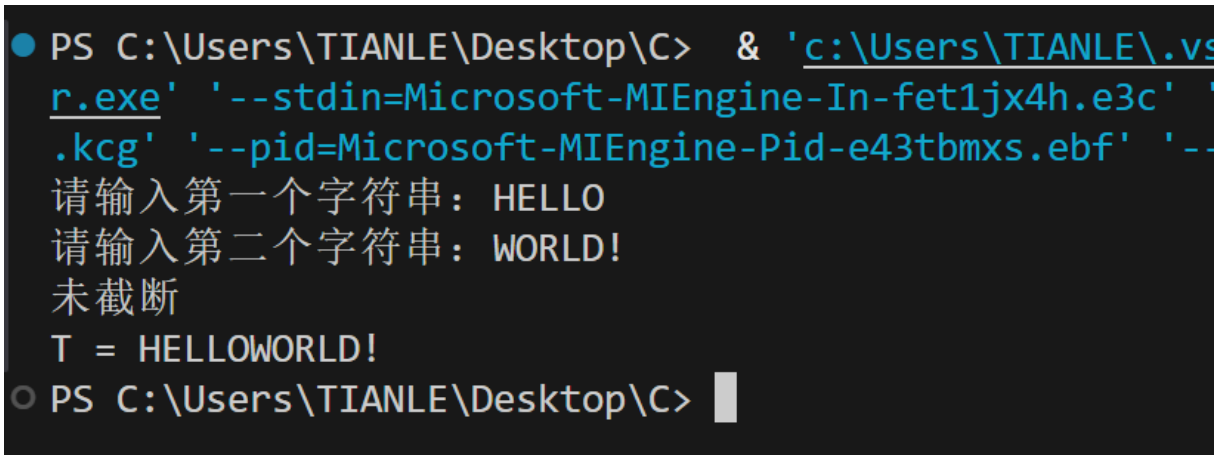
    printf("T = ");

    for (i = 1; i <= T[0]; i++)
        printf("%c", T[i]);
```

```
    printf("\n");

    return 0;
}
```

2.2 输出结果



2.3 实验总结

通过本次实验，我掌握了定长顺序串的存储方式以及字符串连接 Concat 函数的实现方法。同时了解了字符串长度超过最大限制时的截断处理，加深了对顺序存储结构的理解。

3 实验 3

实验任务：串的模式匹配算法 (BF,KMP 并进行计时测试)
实验时间：2 hours
知识点：BF（朴素模式匹配）从主串的每个位置依次与模式串进行比较，匹配失败后回溯并重新开始匹配。KMP 算法利用模式串中已经匹配的信息，通过 next 数组减少不必要的字符比较。BF 实现简单，但可能进行较多重复比较；KMP 通过避免回溯提高匹配效率。

4 源代码

```
#include <stdio.h>
#include <string.h>
#include <time.h>
#define MAXSTRLEN 255  // 用户预定义最大串长
typedef unsigned char SString[MAXSTRLEN + 1];
// 0号单元存放串的长度
#define TRUE 1
#define FALSE 0
typedef int Status;
/*
 * BF 朴素模式匹配算法
*/
int Index(SString S, SString T, int pos)
{
    int i = pos;
    int j = 1;

    while (i <= S[0] && j <= T[0])
    {
        if (S[i] == T[j])
        {
            ++i;
            ++j;
        }
        else
        {
            i = i - j + 2;
            j = 1;
```

```
        }
    }

    if (j > T[0])
        return i - T[0];
    else
        return 0;
}


/*
 * 计算模式串T的 next[] 数组
 */
void get_next(SString T, int *next)
{
    int i = 1;
    next[1] = 0;
    int j = 0;

    while (i < T[0])
    {
        if (j == 0 || T[i] == T[j])
        {
            ++i;
            ++j;
            next[i] = j;
        }
        else
        {
            j = next[j];
        }
    }
}


/*
 * KMP 模式匹配算法
 */
int Index_KMP(SString S, SString T, int pos)
{
```

```
    int next[MAXSTRLEN];

    int i = pos;
    int j = 1;

    get_next(T, next);

    while (i <= S[0] && j <= T[0])
    {
        if (j == 0 || S[i] == T[j])
        {
            ++i;
            ++j;
        }
        else
        {
            j = next[j];
        }
    }

    if (j > T[0])
        return i - T[0];
    else
        return 0;
}


int main()
{
    SString S, T;
    char inputS[MAXSTRLEN + 1];
    char inputT[MAXSTRLEN + 1];

    int i;
    int pos_BF;
    int pos_KMP;

    clock_t start, end;
    double time_BF, time_KMP;
```

```
    /*
     * 输入主串
     */
    printf("请输入主串: ");
    scanf("%255s", inputS);

    /*
     * 输入模式串
     */
    printf("请输入模式串: ");
    scanf("%255s", inputT);


    /*
     * 将输入的普通字符串存入SString
     * S[0]、T[0]存放字符串长度
     */
    S[0] = strlen(inputS);

    for (i = 0; i < S[0]; i++)
    {
        S[i + 1] = inputS[i];
    }

    T[0] = strlen(inputT);

    for (i = 0; i < T[0]; i++)
    {
        T[i + 1] = inputT[i];
    }


    /*
     * BF 模式匹配
     */
    pos_BF = Index(S, T, 1);


    /*
     * KMP 模式匹配
```

```
 */
pos_KMP = Index_KMP(S, T, 1);


/*
 * 输出匹配结果
 */
printf("\n========== 模式匹配结果 ==========\n");

printf("BF匹配位置: %d\n", pos_BF);
printf("KMP匹配位置: %d\n", pos_KMP);


/*
 * BF 计时测试
 *
 * 为了让运行时间更加明显, 重复执行100000次
 */
start = clock();

for (i = 0; i < 100000; i++)
{
    Index(S, T, 1);
}

end = clock();

time_BF = (double)(end - start) / CLOCKS_PER_SEC;


/*
 * KMP 计时测试
 *
 * 重复执行100000次
 */
start = clock();

for (i = 0; i < 100000; i++)
{
    Index_KMP(S, T, 1);
```

```
    }

    end = clock();

    time_KMP = (double)(end - start) / CLOCKS_PER_SEC;


    /*
     * 输出计时结果
     */
    printf("\n========== 计时测试 ==========\n");

    printf("BF运行时间: %.6f 秒\n", time_BF);
    printf("KMP运行时间: %.6f 秒\n", time_KMP);


    return 0;
}
```

4.1 输出结果

```
PS C:\Users\TIANLE\Desktop\C>  & 'c:\Users\TIANLE\.vscode\extensio
r.exe' '--stdin=Microsoft-MIEngine-In-rh3y03b1.ijp' '--stdout=Micr
.pnj' '--pid=Microsoft-MIEngine-Pid-shtroimz.psc' '--dbgExe=D:\Min
请输入主串: abababababababababababc
请输入模式串: abababc

========== 模式匹配结果 ==========
BF匹配位置: 17
KMP匹配位置: 17

========== 计时测试 ==========
BF运行时间: 0.011000 秒
KMP运行时间: 0.008000 秒
```

```
请输入主串：dfjdflafkjahdfoeaodmaidwqalqpofpwfjqjwijoiqwqdjlksajdiwqjasdjlksdjwqiojdsa
请输入模式串：pwf

========== 模式匹配结果 ==========
BF匹配位置：32
KMP匹配位置：32

========== 计时测试 ==========
BF运行时间：0.007000 秒
KMP运行时间：0.016000 秒
```

4.2 实验总结

通过本次实验，我掌握了 BF 朴素模式匹配和 KMP 模式匹配算法的基本原理及实现方法。通过计时测试发现，不同字符串数据会导致两种算法的实际运行时间存在差异。对于重复字符较多的字符串，KMP 能够减少重复比较，运行效率较高；而对于随机字符串，BF 有时反而更快。通过本次实验，加深了对字符串模式匹配算法及其运行效率的理解。